Relatório do Projeto 1

Busca e Organização de Dados

SCC0202 — Algoritmos e Estruturas de Dados I

Integrantes:

17881457 – Bernardo Schramm Schutz Machado

17899853 – João Pedro Cremasco Luiz

15580336 – Thomas Riki Hashima

Parte 1: registros de aproximação de asteroides (`asteroid_dataset_20251019.csv`, 89.227 registros).
Parte 2: registros de jogos publicados na Steam (`games.csv`, 125.855 registros).
Ambiente: Linux x86_64, Intel Core i5-10210U (4 núcleos, 8 threads, 15 GiB de RAM), GCC com `-std=c99 -Wall -Wextra -pedantic`. O executável `trabalho` é gerado por `make`.

1 Organização do código

O programa lê um CSV, guarda cada linha como um ITEM na estrutura escolhida por `--base`, percorre essa estrutura aplicando a consulta, insere os itens aceitos na estrutura escolhida por `--result` e exporta até `top-k` registros. O fluxo é: leitor → ITEM → base → busca com a consulta → resultado → exportação → CSV.

Tabela 1 — Módulos do programa e suas responsabilidades.

Módulo	Responsabilidade
`src/item.c`	ITEM: conjunto de pares nome da coluna/valor em texto, independente do domínio. ITEM_STORE: dono de todos os itens criados.
`src/reader.c`	Lê o cabeçalho, detecta o delimitador (vírgula, ponto e vírgula ou tabulação), trata campos entre aspas e vazios e cria os itens. É o único módulo específico de cada dataset (`--reader nasa`, `--reader games`).
`src/query.c`	Interpreta `FILTER`, `SIM`, `ACCEPT`, `ORDER`, `GROUP` e `CROSS`; decide se um item é aceito e calcula a similaridade.
`src/collection.c`	Interface comum às oito estruturas (criar, inserir, percorrer e destruir), com um `switch` sobre o tipo escolhido.
`src/export.c`	Escreve o CSV na ordem e no delimitador da entrada; em jogos, usa o cabeçalho corrigido para separar `Discount` e `DLC count`.
`src/main.c`	Argumentos, medição de tempo, orquestração e saída padrão.
`data_structures/`	Oito TADs, um por arquivo, cada um com o seu nó ou vetor.

Os itens são guardados como texto e convertidos para número apenas quando uma consulta pede (`strtod`). Assim, valores ausentes não interrompem a leitura e o arquivo de saída reproduz o valor original.

2 Estruturas de dados

Todas as estruturas foram implementadas pelo grupo, sem contêineres de bibliotecas externas. Elas não conhecem o conteúdo dos itens: guardam ponteiros genéricos, e o módulo `collection` converte para ITEM em um único ponto.

Os nomes do enunciado (`pilha`, `fila`, `lista_seq`, `lista_enc`, `lista_cabeca`, `lista_ord`, `lista_gen`, `lista_cruz`) também são aceitos.

Tabela 2 — Estruturas implementadas e o que diferencia cada uma.

Estrutura	Argumento	Implementação	O que a diferencia
Pilha	`stack`	vetor redimensionável	Inserção e remoção no topo; percurso do topo para a base.
Fila	`queue`	nós encadeados	Inserção no fim, remoção no início; percurso da frente para o fim.
Lista sequencial	`array_list`	vetor redimensionável	Acesso por índice; a remoção desloca elementos.
Lista encadeada	`linked_list`	nós duplamente encadeados, ponteiros para o primeiro e o último	Inserção no fim em tempo constante; remoção em qualquer posição.
Lista com cabeça	`sentinel_list`	encadeada com sentinela real	O sentinela não representa registro; inserção, busca, remoção e percurso partem dele.
Lista ordenada	`ordered_list`	vetor mantido em ordem	A posição é localizada por busca binária e o elemento é inserido já na ordem de `ORDER`, sem ordenar ao final.
Lista generalizada	`generalized_list`	átomos e sublistas	`GROUP` cria grupos e subgrupos (por exemplo, categoria e depois ano); o percurso é em profundidade e visita apenas os itens.
Lista cruzada	`crossed_list`	nós esparsos por par de chaves, ligados nas dimensões	A coleção guarda em cada célula os itens que compartilham o par de valores de `CROSS`.

3 Consultas e exemplos de execução

A consulta é um arquivo texto com campos separados por `|`. `FILTER` define condições obrigatórias (`==`, `!=`, `<`, `<=`, `>`, `>=`; numéricas quando os dois valores são números). `SIM` define um critério de similaridade

$$s(x, q, t) = \max\left(0,\ 1 - \frac{|x - q|}{t}\right),$$

e a similaridade total é a média ponderada dos critérios. `ACCEPT` define o limiar mínimo. `ORDER`, `GROUP` e `CROSS` configuram a lista ordenada, a generalizada e a cruzada.

3.1 Parte 1 — Asteroides

Exemplo 1 — filtro e similaridade. Consulta `queries/asteroids_by_risk.txt`:

```
FILTER|distance_au|<=|0.08
SIM|velocity_km_s|15.0|10.0|1.0
SIM|risk_score|0.70|0.30|2.0
ACCEPT|0.60
```

```
./trabalho --reader nasa --input asteroid_dataset_20251019.csv --base linked_list --result
    ↪ array_list --query queries/asteroids_by_risk.txt --output resultados/asteroids_by_risk.csv --
    ↪ top-k 100 --limit 10000
```

Exemplo 2 — resultado ordenado. Consulta `queries/asteroids_ordered.txt`:

```
FILTER|distance_au|<=|0.08
ORDER|risk_score|DESC
```

```
./trabalho --reader nasa --input asteroid_dataset_20251019.csv --base queue --result ordered_list --
    ↪ query queries/asteroids_ordered.txt --output resultados/asteroids_ordered.csv --top-k 50 --
    ↪ limit 10000
```

Exemplo 3 — agrupamento e índice cruzado. Consulta `queries/asteroids_group_cross.txt`:

```
FILTER|distance_au|<=|0.1
GROUP|threat_category|year
CROSS|threat_category|year
```

```
./trabalho --reader nasa --input asteroid_dataset_20251019.csv --base generalized_list --result
    ↪ crossed_list --query queries/asteroids_group_cross.txt --output resultados/
    ↪ asteroids_group_cross.csv --top-k 100 --limit 20000
```

3.2 Parte 2 — Jogos

Os mesmos TADs são reutilizados, e apenas o leitor muda (`--reader games`). Esse leitor separa os nomes `Discount` e `DLC count`, que aparecem unidos no cabeçalho do arquivo fornecido.

Exemplo 4 — agrupamento por gênero e idade mínima. Consulta `queries/games_by_genre.txt`:

```
FILTER|Windows|==|true
FILTER|Price|>=|1
FILTER|Price|<=|100
SIM|Peak CCU|500|1000|1
ACCEPT|0.4
GROUP|Genres|Required age
```

```
./trabalho --reader games --input games.csv --base linked_list --result generalized_list --query
    ↪ queries/games_by_genre.txt --output resultados/games_by_genre.csv --top-k 100 --limit 10000
```

Exemplo 5 — índice cruzado. Consulta `queries/popular_games_by_genre.txt`:

```
FILTER|Peak CCU|>=|1000
FILTER|Windows|==|true
CROSS|Genres|Windows
```

```
./trabalho --reader games --input games.csv --base array_list --result crossed_list --query queries/
    ↪ popular_games_by_genre.txt --output resultados/popular_games_by_genre.csv --top-k 100 --
    ↪ limit 10000
```

Exemplo 6 — outra combinação de TADs. Consulta `queries/popular_games_ordered.txt`:

```
FILTER|Windows|==|true
FILTER|Peak CCU|>=|1000
ORDER|Peak CCU|DESC
```

```
./trabalho --reader games --input games.csv --base array_list --result ordered_list --query queries/
    ↪ popular_games_ordered.txt --output resultados/popular_games_ordered.csv --top-k 100 --limit
    ↪ 10000
```

3.3 Resultados dos seis exemplos

Os seis exemplos foram reexecutados três vezes nesta auditoria; as medições individuais estão em `resultados/examples_raw.csv`. Os arquivos de asteroides mantêm o cabeçalho original. O arquivo de jogos tem o nome `DiscountDLC count` unido no cabeçalho, embora os registros tenham duas colunas; o leitor o separa em `Discount` e `DLC count`, e a exportação usa esses nomes corrigidos. A ordem das colunas e o delimitador são mantidos.

Tabela 3 — Resultados dos exemplos de execução (médias de três execuções, tempos em ms).

Ex.	Base → resultado	Lidos	Enc.	Exp.	Carga	Busca	Ins. res.	Escrita
1	`linked_list→array_list`	10.000	93	93	91,794	13,293	0,010	0,853
2	`queue→ordered_list`	10.000	5.501	50	74,342	15,694	85,696	0,490
3	`generalized_list→crossed_list`	20.000	12.919	100	137,972	29,708	6,710	0,845
4	`linked_list→generalized_list`	10.000	6.145	100	512,497	21,991	15,895	4,561
5	`array_list→crossed_list`	10.000	43	43	524,614	4,484	0,065	2,738
6	`array_list→ordered_list`	10.000	43	43	494,867	7,803	0,100	2,653

4 Memória

Os itens pertencem ao ITEM_STORE. As estruturas guardam ponteiros emprestados e liberam apenas o que alocam para se organizar (nós, vetores e índices). Assim, o mesmo item pode estar ao mesmo tempo na base, no resultado e nos dois índices da lista cruzada, sem cópia e sem liberação dupla. No fim da execução, as coleções são destruídas primeiro e o store por último.

Os seis exemplos executados com AddressSanitizer e UndefinedBehaviorSanitizer terminaram sem erros reportados, com a detecção de vazamentos do LeakSanitizer desativada por incompatibilidade do ambiente. Valgrind não encontrou erros nem vazamentos nos exemplos 1, 3, 4 e 5, nem na suíte dos TADs.

5 Complexidade assintótica

Seja n o número de itens armazenados, k o número de registros exportados, p o número de campos por item, g o número de grupos, s o de subgrupos examinados, m o de itens no subgrupo, r e c o número de chaves distintas dos índices da lista cruzada, h os itens no bucket da célula e d_r e d_c os graus da linha e da coluna percorridas.

Carregar n itens custa $O(n)$ amortizado em pilha, fila e listas simples. Na lista ordenada pode custar $O(n^2)$ pelos deslocamentos, e na generalizada e na cruzada depende da quantidade de chaves distintas e dos graus. A busca da consulta percorre os n itens da base independentemente da estrutura. Como `item_get` percorre linearmente os p campos do item, o custo é $O(n(f+q)p)$ para f filtros e q critérios `SIM`, mais a inserção dos aceitos e a gravação do campo `similarity`, que acrescenta um campo ao item. A exportação custa $O(kp^2)$, pois cada uma das p colunas é procurada no item, mais a escrita dos caracteres.

Tabela 4 — Complexidade das operações realmente usadas pelo programa.

Estrutura	Inserção	Busca / percurso	Remoção
Pilha	$O(1)$ amortizado; $O(n)$ ao redimensionar	$O(n)$	$O(1)$ no topo; localizar item arbitrário $O(n)$
Fila	$O(1)$	$O(n)$	$O(1)$ na frente; localizar item arbitrário $O(n)$
Lista sequencial	$O(1)$ amortizado no fim	$O(n)$	$O(n)$, pois busca e desloca
Lista encadeada	$O(1)$ no fim (ponteiro final)	$O(n)$	$O(n)$ para localizar; $O(1)$ para religar
Lista com cabeça	$O(1)$ no fim	$O(n)$	$O(n)$ para localizar; $O(1)$ para religar
Lista ordenada	$O(n)$: busca binária $O(\log n)$ mais deslocamento $O(n)$	$O(\log n)$ por chave; $O(n)$ para percorrer	$O(n)$
Lista generalizada	$O(g+s)$ para achar ou criar grupo e subgrupo; $O(1)$ para anexar	$O(n)$	$O(g+s+m)$ no pior caso
Lista cruzada	$O(r+c+d_r+d_c)$ para célula nova; $O(r+c)$ para anexar em célula existente	$O(n)$ total; $O(d_r)$ ou $O(d_c)$ por linha ou coluna	$O(r+c+h)$ para localizar chaves e item no bucket

6 Avaliação experimental

6.1 Protocolo

- Base `linked_list` e resultado `array_list`, nos dois datasets, com as consultas `queries/asteroids_benchmark.txt` (`FILTER|distance_au|<=|0.1`) e `queries/games_benchmark.txt` (`FILTER|Windows|==|true`).
- Seis pares de `--limit` e `--top-k`: (1.000, 10), (5.000, 50), (10.000, 100), (25.000, 500), (50.000, 1.000) e (0, 0), em que zero significa toda a base e todos os resultados.
- Cada configuração teve uma execução de aquecimento, descartada, e três execuções medidas. As tabelas mostram a média aritmética e o desvio padrão amostral ($\pm$). As 36 medições individuais estão em `resultados/benchmark_raw.csv`.
- A coluna N é o número de registros lidos (`INPUT_RECORDS`).

6.2 Metodologia de medição

Os tempos são medidos dentro do próprio programa com `clock()` de `time.h` (C99), que devolve o tempo de CPU do processo, usuário mais sistema, em ticks de 1/`CLOCKS_PER_SEC` (1 µs no Linux), convertido para milissegundos. Cada fase é delimitada por duas leituras do relógio. Como o relógio mede CPU e não tempo decorrido, esperas por disco não entram nas medições, e os valores de carga e de saída não equivalem à latência percebida pelo usuário.

A busca e a inserção no resultado são feitas em duas passadas. Na primeira, `collection_visit` avalia os critérios de cada item, grava o campo `similarity` e guarda o ponteiro dos itens aceitos em um vetor. Na segunda, um laço insere todos eles na estrutura do resultado, com um único par de chamadas a `clock()` ao redor do laço inteiro. Assim, a busca não inclui a inserção, e o relógio não é lido uma vez por item. Não entram nas medições a abertura do arquivo de entrada, a leitura da consulta, a criação e a destruição das estruturas e a inicialização do processo, e por isso a coluna Total é a soma das quatro fases, e não o tempo de parede do programa.

A medição por item faria duas leituras de `clock()` para cada inserção e repetiria esse custo muitas vezes. Por isso, o programa mede o laço inteiro em bloco.